

Spring Boot Microservices with MongoDB

Learning Outcomes

After completing this lab, students will be able to:

- Create microservices using Spring Boot
- Implement REST APIs
- Use MongoDB Atlas as the database
- Apply the Database-per-Service pattern
- Test APIs using Postman
- Prepare services for Eureka-based service discovery in future labs

Scenario

Suppose you are building an Online Shop using Microservice Architecture.

For simplicity, consider that the system consists of the following microservices:

1. Employee Service
2. Customer Service
3. Product Service
4. Order Service

Each microservice will:

- Have its own MongoDB database
- Expose REST APIs
- Run independently on a different port

Required Software

- IntelliJ IDEA
- Java 17
- Maven
- MongoDB Atlas
- Postman

Part 0: MongoDB Atlas Setup

Before creating the microservices, configure MongoDB Atlas.

Step 1: Create a MongoDB Atlas Account

1. Visit <https://www.mongodb.com/atlas>
2. Sign up or log in.
3. Create a new project.

Example project name: Microservices Lab

Step 2: Create a Cluster

1. Create a new project.
2. Select the Free Tier option.
3. Select authentication using username and password.
4. Choose a password you can remember and create a user.
5. Add 0.0.0.0/0 to the IP Access List.
6. Finish creating the cluster.

Step 3: Obtain the MongoDB Connection String

1. Open the cluster → Connect
2. Select: Drivers
3. Choose Java from the list of drivers.
4. Copy and save the connection string in a notepad.

Atlas will generate a connection string similar to:

[mongodb+srv://student:<password>@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority](#)

Replace: <password> with your database user's password.

Example:

[mongodb+srv://student:student123@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority](#)

Part 1: Create Spring Boot Project

Create a spring-boot application using spring-boot [initializr](#) using Maven, Java 17, Spring Boot 4.0.6.

Name: employee-service

Dependencies

- Spring Web
- Spring Data MongoDB
- Lombok
- Eureka Discovery Client

Note: Eureka Discovery Client should be added now, but Eureka functionality will remain disabled in this lab.

Extract the .zip and open the extracted folder in IntelliJ Idea.

In src.main.java.package create the following packages

- controller
- entity
- repository
- service

In the entity package, create a class called Employee with the following attributes: id, name, designation, salary. Annotate the class properly.

```
@Document(collection = "employees")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Employee {

    @Id
    private String id;
    @Field
    private String name;
    @Field
    private String designation;
    @Field
    private double salary;
```

```
}
```

In repository package, create `EmployeeRepository` and add the following code:

```
public interface EmployeeRepository extends MongoRepository<Employee, String> {  
  
    Employee findEmployeeById(String userId);  
}
```

In service package, create `EmployeeService` class and add the following code:

```
@Service  
public class EmployeeService {  
    @Autowired  
    private EmployeeRepository employeeRepository;  
  
    public Employee saveEmployee(Employee employee) { return  
employeeRepository.save(employee); }  
  
    public Employee findEmployeeById(String userId) { return  
employeeRepository.findEmployeeById(userId); }  
}
```

In the controller package, create a controller namely `EmployeeController`. Here, rest methods will be implemented for employee. You can add the following code:

```
@RestController  
@RequestMapping("/employees")  
public class EmployeeController {  
    @Autowired  
    public EmployeeService employeeService;  
  
    @PostMapping("/")  
    public Employee saveEmployee(@RequestBody Employee employee) {  
        return employeeService.saveEmployee(employee);  
    }  
  
    @GetMapping("/{id}")  
    public Employee findEmployeeById(@PathVariable("id") String employeeId) {  
        return employeeService.findEmployeeById(employeeId);  
    }  
}
```

In `EmployeeServiceApplication`, add the following code:

```
@SpringBootApplication
public class EmployeeServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmployeeServiceApplication.class, args);
    }

}
```

Locate the application.properties file in resources. Add the following and replace with your own username and password:

```
spring.mongodb.uri=mongodb+srv://username:password@cluster0.7kmjsln.mongodb.net
/?appName=Cluster0
spring.mongodb.database=employee_db
```

Note:

MongoDB Atlas will automatically create the database and collections when the first document is inserted. You do not need to manually create employee_db, customer_db, or product_db beforehand.

Add an application.yml file under resources and add:

```
server:
  port: 9001
spring:
  application:
    name: EMPLOYEE-SERVICE
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    hostname: localhost
```

Now run EmployeeServiceApplication and test the APIs in Postman.

Similarly, create Customer, Product and Order microservices on three other different ports so that they can be run simultaneously. Replace `employee_db` with appropriate names respectively in the `application.properties` files to make sure each microservice has its own database.

Product entity consists of id, name, description, price.

Customer entity consists of id, name, address, age.

Order entity consists of id, customerId, productId, quantity, employeeId, date, etc.

Keep all microservices under a common parent folder.